CSP-203: Software Tools & Technologies Lab

Chapter – 2

Build Tool: Make (Makefile) and Cmake (CMakeLists.txt)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- Bash Script (C program execution)
- Make
  - Makefile
- Cmake
  - CMakeLists.txt

- **Automated compilation and execution of C program file:**
  - write gcc file_name.c -o file_name
  - Then append ./file_name

```
#! /bin/bash

gcc hello.c -o hello
./hello
~
```

```
arindam@ARINDAM:~$ cat hello.c
#include<stdio.h>

void main(){
        printf("Hello User!!!!\n");
}
arindam@ARINDAM:~$ ./demo.sh
Hello User!!!!
arindam@ARINDAM:~$
```

❑ Automated compilation and execution of C program file over different inputs:

➢ Example:

```
arindam@ARINDAM:~$ cat hello.c
#include<stdio.h>
#include<stdlib.h>

void main(int argc, char* argv[]){
        if(argc<2)
        {
                printf("Input not provided!!!");
                exit(1);
        }
        int a = atoi(argv[1]);
        printf("Your entered number is: %d\n",a);
}
arindam@ARINDAM:~$ ./demo.sh
Executing for loop 0:
Your entered number is: 12
Executing for loop 1:
Your entered number is: 13
Executing for loop 2:
Your entered number is: 41
Executing for loop 3:
Your entered number is: 20
Executing for loop 4:
Your entered number is: 55
arindam@ARINDAM:~$
```

```
#! /bin/bash

array=(12 13 41 20 55)
gcc hello.c -o hello
for i in ${!array[@]}
do
        echo "Executing for loop $i:"
        ./hello ${array[$i]}
done
```

- **Automated compilation and execution of C program file over different inputs:**
  - Take two parameters in main fuction, argc and char *argv[]
  - argc calculates the number of arguments given to the main function.
  - argv array stores the strings passed from the bash script.
  - Bash script passes everything as a string.
  - Now we have to convert the string to integer/float value in c file.

- **Converting a number present as string to integer:**
  - There is a standard library in C.
  - Inclusion of the header file is like this: #include<stdlib.h>
  - Standard library contains atoi() function.
  - We can use this function to convert a number in string to integer value.
  - integer num = atoi(string_num);

- **Converting a number present as string to float:**
  - There is a standard library in C.
  - Inclusion of the header file is like this: #include<stdlib.h>
  - Standard library contains atof() function.
  - We can use this function to convert a number in string to float value.
  - float num = atof(string_num);

❑ Automated compilation and execution of C program file over different inputs:

➢ Example:

```
arindam@ARINDAM:~$ cat hello.c
#include<stdio.h>
#include<stdlib.h>

void main(int argc, char* argv[]){
        if(argc<2)
        {
                printf("Input not provided!!!");
                exit(1);
        }
        int a = atoi(argv[1]);
        printf("Your entered number is: %d\n",a);
}
arindam@ARINDAM:~$ ./demo.sh
Executing for loop 0:
Your entered number is: 12
Executing for loop 1:
Your entered number is: 13
Executing for loop 2:
Your entered number is: 41
Executing for loop 3:
Your entered number is: 20
Executing for loop 4:
Your entered number is: 55
arindam@ARINDAM:~$
```

```
#! /bin/bash

array=(12 13 41 20 55)
gcc hello.c -o hello
for i in ${!array[@]}
do
        echo "Executing for loop $i:"
        ./hello ${array[$i]}
done
```

Manual Compilation

- Example – manual compilation:

- `main.cpp`
- `class1.cpp`
- `class2.cpp`

- `$ g++ main.cpp class1.cpp class2.cpp -o main`
- `$ ./main`

- Manually compile all files using gcc from the command line.

Make

- The `make` utility is a commonly used build tool that can manage the process of building and rebuilding a project.
- Make was originally written in 1976 by Stuart Feldman, who received the 2003 ACM Software System award for his work.

Make utility

Command line utility used to process the instructions written in *Makefile*.

```
make - GNU make utility to maintain groups of programs

SYNOPSIS
       make [OPTION]... [TARGET]...

DESCRIPTION
       The  make  utility will determine automatically which pieces of a large program need to be recompiled, and issue the commands to recompile them.  The manual
       describes the GNU implementation of make, which was written by Richard Stallman and Roland McGrath, and is currently maintained by Paul Smith.  Our examples
       show  C  programs,  since they are very common, but you can use make with any programming language whose compiler can be run with a shell command.  In fact,
       make is not limited to programs.  You can use it to describe any task where some files must be updated automatically from others whenever the others change.

       To prepare to use make, you must write a file called the makefile that describes the relationships among files in your program, and the states the  commands
       for updating each file.  In a program, typically the executable file is updated from object files, which are in turn made by compiling source files.

       Once a suitable makefile exists, each time you change some source files, this simple shell command:

              make

       suffices  to  perform  all  necessary recompilations.  The make program uses the makefile description and the last-modification times of the files to decide
       which of the files need to be updated.  For each of those files, it issues the commands recorded in the makefile.

       make executes commands in the makefile to update one or more target names, where name is typically a program.  If no -f option is present,  make  will  look
       for the makefiles GNUmakefile, makefile, and Makefile, in that order.

       Normally you should call your makefile either makefile or Makefile.  (We recommend Makefile because it appears prominently near the beginning of a directory
       listing, right near other important files such as README.)  The first name checked, GNUmakefile, is not recommended for most makefiles.  You should use this
       name  if  you have a makefile that is specific to GNU make, and will not be understood by other versions of make.  If makefile is '-', the standard input is
       read.

       make updates a target if it depends on prerequisite files that have been modified since the target was last modified, or if the target does not exist.
```

Usage: make **<options> <targert>**

Common Options:

- **-f**: used to use a file as a makefile
- **-d**: print the debugging information along with normal processing
- **-i**: used to ignore all errors in commands

```
#!/bin/bash
make
```

Makefile

- *Makefile* is a script that automates compiling and linking.
- Essential in software development for efficient build.
- Commonly used to build C/C++ targets.

1 target: dependencies



$ gcc hello.c -o hello

- target: What to build; e.g., an executable
- dependencies: Files needed for building

2 <Tab> command (or recipe)

- command: Action to perform the build and generate the executable
  - <compiler> <flags> <source files> -o <target>

Don't forget to include a <tab> character before every target recipe!

Why Makefile?

- **For example, 4 source files required to compile at the same time; main.c, hello.c, action.c and functions.h**

Example: gcc main.c hello.c action.c -o executable

Note

But if there are 1000s of source code files to be compiled at the same time, do we need to write manually all the files one by one ?

NO!

Simple Solution: With make and Makefile you only need to execute **make**

Common Makefile Macros

Variable	***Variable Description***
$@	The name of the executable file to be made
$?	The name of the changed dependencies
CC	Program to compile C programs, default is `gcc’
CXX	Program to compile C programs, default is `g++’
CFLAGS	Extra flags to be passed to C compiler
CXXFLAGS	Extra flags to be passed to C++ compiler
LDFLAGS	Extra flags to give to compilers when they are supposed to invoke the linker

Create executable using Makefile Macros

```
CC = gcc
CFLAGS = -Wall -O2
myprogram: main.c a.c
        $(CC) $(CFLAGS) -o $@  main.c  a.c
```

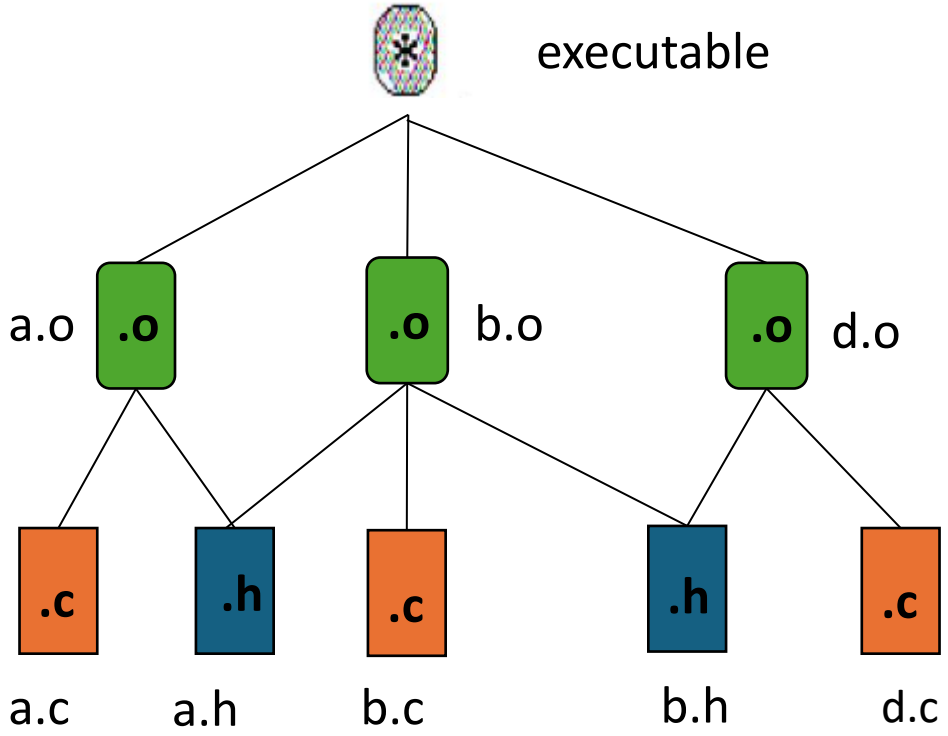
- **<target: dependencies>**
- **<compiler> <flags> -o <target> <source files>**

PS:

-wall : Stands for “Warning all” and instructs the compiler to enable a comprehensive set of warning messages.

-O2 : Stands for “Optimization Level 2” and instructs the compiler to apply a set of optimizations to the generated machine code

Writing a Makefile with dependencies



Sample Makefile

```
executable: a.o b.o d.o
        gcc a.o b.o  d.o -o executable

a.o:  a.c a.h
        gcc  -c  a.c

b.o:  a.h b.h b.c
        gcc  -c b.c

d.o:  b.h  d.c
        gcc  -c d.c
```

Rules and Targets in Makefile

- Rules define how to build targets
- Rules include prerequisites and a recipe
- Prerequisites are the dependencies for the target
- Recipe contains the commands to build the target

Example:

```
all: target
target: main.c helper.c
        gcc -o target main.c helper.c
```

<rules>: target/custom targets

```
target: <prerequisites>
        gcc -o target main.c helper.c  ⟵ <recipe>
```

Make

- Example – manual compilation:

- `main.cpp`
- `class1.cpp`
- `class2.cpp`

```
$ g++ main.cpp class1.cpp class2.cpp -o main
$ ./main
```

- Manually compile all files using gcc from the command line.

Make

- Example – simple makefile:

```
all:
	g++ main.cpp class1.cpp class2.cpp -o main
```

```
$ g++ main.cpp class1.cpp class2.cpp -o main
$ ./main
```

- Makefile with a single (default) target that compiles all the files.

Make

- Example – Makefile with dependencies

```
all: main
main: main.o class1.o class2.o
	g++ main.o class1.o class2.o -o main
main.o: main.cpp
	g++ -c main.cpp
class1.o:
	g++ -c class1.cpp
class2.o:
	g++ -c class2.cpp
```

- Makefile that considers each file and dependencies and recompiles only if necessary.

Make

- Example – Makefile with clean

```
clean:
	rm -rf *.o main
```

```
$ make clean
$ make
$ ./main
```

- Makefiles often contain a clean target that can be used to completely clean the project of all compiled files and completely rebuild from scratch.

Make

- Example – Makefile using Macros

```
# Comment – Selecting g++ as the compiler
CC=g++
all: main
main: main.o class1.o class2.o
	$(CC) main.o class1.o class2.o –o main
main.o: main.cpp
	$(CC) -c main.cpp
...
```

- Macros can be used in Makefiles to easily switch between different options and avoid rewriting large sections.

Sample Makefile

CC = gcc → specifies the C compiler
CFLAGS = -Wall -O2 → holds the compilation flags, including `-Wall' for warnings and `-O2' for optimizations

LDLIBS = -lm → specifies any libraries that needs to be linked, here `-lm' which is the math library

SOURCES = main.c helper.c → lists the source files

TARGET = myprogram → hold the name of the executable

all: $(TARGET) → this target depends on $(TARGET) which will be built if necessary and builds program from scratch if run make all

$(TARGET): $(SOURCES) → specifies the compilation rule to build the target
```
    $(CC) $(CFLAGS) -o $(TARGET) $(SOURCES) $(LDLIBS)
```

clean: → removes the generated files if run make clean
```
    rm -f $(TARGET)
    rm –f *.o
```

.PHONY: all clean → lists targets that don't represent files to be built like 'all' and 'clean'

Makefile: Example#1

Filename: *Makefile*

```
CC = gcc
CFLAGS = -Wall
hello: client.c server.c hello.h
	$(CC) $(CFLAGS) -o $@  client.c server.c
```

Execution: $ make –f *Makefile*

Makefile: Example#2

Filename: *Makefile*

```
CC = gcc
CFLAGS = -Wall
hello: main.c factorial.c hello.c functions.h
	$(CC) $(CFLAGS) -o $@  main.c factorial.c hello.c
```

Execution: $ make –f *Makefile*

Makefile: Example#3

Filename: *Makefile*

```
CC = gcc
CFLAGS = -Wall

TARGET = result
all: $(TARGET)
$(TARGET): main.c functions.c hello.c functions.h
	$(CC) $(CFLAGS) -o $@ main.c functions.c hello.c functions.h
clean:
	rm -f  $(TARGET)

.PHONY: all clean
```

Execution: $ make –f *Makefile*

$ make or $ make all
$ make clean

Makefile: Example#4

Filename: *Makefile*

```
CC = gcc
CFLAGS += -Wall
CFLAGS += -O2
TARGET = math_program
LDLIBS = -lm
all: $(TARGET)
$(TARGET): main.o hello.o
	$(CC) main.o hello.o -o $@ $(LDLIBS)
main.o: main.c functions.h
	$(CC) -c  $(CFLAGS)  main.c $(LDLIBS)
hello.o: hello.c
	$(CC) -c  $(CFLAGS) hello.c
clean:
	rm -f  *.o $(TARGET)
.PHONY: all clean
```

Execution: $ make –f *Makefile*



Make

•For more information see:

•http://www.gnu.org/software/make/manual/make.html

CMake

•CMake is the cross-platform, open-source build system designed to control the software compilation using platform and compiler independent configuration files.

•This tool automatically generates build scripts for different operating systems - Visual Studio projects for Windows and makefiles for Unix/Linux.

CMake generates a Makefile using CMakeLists.txt.

CMake

- CMake generates build scripts from files named:

- `CMakeLists.txt`

CMake

- Example – Simple CMake
- `cmake_minimum_required(VERSION 3.22.1)`
- `project(Main)`
- `add_executable(Main main.cpp class1.cpp class2.cpp)`

How to a Makefile

- $ mkdir build
- $ cd build
- $ cmake ..

OR

- Cmake –Bbuild -> This will automatically create a build folder and will run cmake

CMake

- Example – Simple CMake

```
$ make
Scanning dependencies of target Main
[ 33%] Building CXX object CMakeFiles/Main.dir/main.cpp.o
[ 66%] Building CXX object CMakeFiles/Main.dir/class1.cpp.o
[100%] Building CXX object CMakeFiles/Main.dir/class2.cpp.o
Linking CXX executable Main
[100%] Built target Main
```

CMake

- For more information on CMake:

- http://www.cmake.org/cmake/help/cmake_tutorial.html

Thank you!

Email: anand@iitbhilai.ac.in